

Università degli Studi di Padova
DIPARTIMENTO DI MATEMATICA “TULLIO LEVI-CIVITA”
CORSO DI LAUREA IN INFORMATICA



**Evoluzione e ottimizzazione di un firmware embedded per
sistema di telemetria basato su ESP32**

Tesi di Laurea

Relatore

Prof. Zanella Marco

Laureando

Laoud Zakaria
Matricola 2113196

إِنَّ مَعَ الْعُسْرِ يُسْرًا

“In verità, con la difficoltà giunge la facilità.”

— Corano(96 - 6)

Ringraziamenti

Desidero esprimere la mia sincera gratitudine al professor Zanella Marco, mio relatore, per l'aiuto e il sostegno che mi ha dato durante la stesura dell'elaborato (da scrivere meglio)

Ricorda di ringraziare il prof cavallin.

Padova, Settembre 2026

Laoud Zakaria

Sommario

Il presente lavoro di tesi riguarda l'evoluzione e l'ottimizzazione di un firmware embedded sviluppato su piattaforma ESP32 per un sistema di telemetria applicato al settore motorsport. L'obiettivo principale è il miglioramento dell'efficienza, dell'affidabilità e delle funzionalità del sistema attraverso interventi sulla gestione dei dati, sull'acquisizione delle informazioni e sull'ottimizzazione delle risorse hardware disponibili.

Il lavoro si concentra sull'analisi del firmware esistente e sull'introduzione di miglioramenti volti a rendere il sistema più performante e adattabile alle esigenze di acquisizione dati in ambito racing. Verranno inoltre integrate nuove funzionalità per ampliare le capacità del dispositivo e migliorare la qualità delle informazioni raccolte durante l'utilizzo.

Indice

1	Introduzione	1
1.1	L'azienda	1
1.2	Il progetto	1
1.3	Motivazioni della scelta del progetto	2
1.4	Convenzioni tipografiche	3
2	Descrizione dello stage	4
2.1	Competenze e obiettivi	4
2.1.1	Competenze da acquisire	4
2.1.2	Obiettivi	5
2.1.2.1	Obiettivi obbligatori	5
2.1.2.2	Obiettivi desiderabili	5
2.1.2.3	Obiettivi facoltativi	5
2.2	Pianificazione	6
2.3	Organizzazione del lavoro	7
2.4	Analisi preventiva dei rischi	8
3	Analisi dei requisiti	9
3.1	Stato attuale del firmware	9
3.2	Analisi degli utenti	9
3.3	Lista degli <i>use case</i>	9
3.3.1	UC1 – Acquisizione binaria dei dati sensore	10
3.3.2	UC2 – Registrazione binaria dei dati sensore su SD	11
3.3.3	UC3 – Acquisizione dati dal bus CAN	12
3.3.4	UC4 – Gestione dello stato del firmware tramite LED_G	13
3.3.4.1	UC4.1 – Segnalazione di pairing WiFi	14
3.3.4.2	UC4.2 – Segnalazione di pairing BLE	14
3.3.4.3	UC4.3 – Segnalazione di dispositivo avviato in attesa di sessione	15
3.3.4.4	UC4.4 – Segnalazione di scheda microSD mancante	15
3.3.4.5	UC4.5 – Segnalazione di logging normale	16
3.3.4.6	UC4.6 – Segnalazione di sospensione per inattività	16
3.3.4.7	UC4.7 – Segnalazione di errore generico	17
3.3.5	UC5 – Configurazione dell' <i>Output Data Rate$_G$</i> (ODR) dei sensori	17
3.3.6	UC6 – Sospensione dell'acquisizione sensori in stato di inattività	18
3.3.7	UC7 – Calibrazione dei sensori	19
3.4	Tracciamento dei requisiti	20
3.4.1	Requisiti funzionali	20
3.4.2	Requisiti non funzionali	21

3.4.3	Requisiti di vincolo	21
3.4.4	Riepilogo dei requisiti	22
4	Tecnologie utilizzate	23
5	Progettazione e codifica	24
5.1	Tecnologie e strumenti	24
5.2	Ciclo di vita del software	24
5.3	Progettazione	24
5.3.1	Namespace 1	24
5.4	Design Pattern utilizzati	24
5.5	Codifica	24
6	Verifica e validazione	25
7	Conclusioni	26
7.1	Consuntivo finale	26
7.2	Raggiungimento degli obiettivi	26
7.3	Conoscenze acquisite	26
7.4	Valutazione personale	26
	Glossario	i
	Bibliografia	v
	Sitografia	vi

Elenco delle figure

1.1	Logo dell'azienda Eggon Srl	1
3.1	Diagramma UML – UC1: Acquisizione binaria dei dati sensore	10
3.2	Diagramma UML – UC2: Registrazione binaria dei dati sensore su SD . . .	11
3.3	Diagramma UML – UC3: Acquisizione dati dal CAN	12
3.4	Diagramma UML – UC4: Gestione dello stato del firmware tramite LED . .	13
3.5	Diagramma UML – UC5: Configurazione dell'ODR dei sensori	17
3.6	Diagramma UML – UC6: Sospensione dell'acquisizione sensori in stato di inattività	18
3.7	Diagramma UML – UC7: Calibrazione dei sensori	19

Elenco delle tabelle

3.1	Tracciamento dei requisiti funzionali.	21
3.2	Tracciamento dei requisiti non funzionali.	21
3.3	Tracciamento dei requisiti di vincolo.	22
3.4	Conteggio dei requisiti tracciati.	22

Elenco dei codici sorgenti

1.1	Fibonacci recursive	3
-----	-------------------------------	---

1. Introduzione

Nel seguente capitolo viene presettata l'azienda ospitante, il progetto svolto e le motivazioni che hanno portato alla scelta di questo argomento per il lavoro di tesi.

1.1 L'azienda

Eggon S.r.l. è una *software house* con sede a Padova che opera in diversi ambiti tecnologici, sviluppando soluzioni software innovative e fornendo servizi personalizzati in base alle esigenze dei propri clienti. L'attività dell'azienda si concentra principalmente in settori quali la *Digital Health_G*, l'*Industria 5.0_G* con dispositivi correlati all'*Industrial Internet of Things_G*, l'*Insurtech_G* e il *Fintech_G*, oltre che nell'ambito dell'automazione logistica.



Figura 1.1: Logo dell'azienda Eggon Srl

La società nasce dall'esperienza maturata negli Stati Uniti dai suoi fondatori, Gianpaolo Ferrarin e Fulvio Menegozzo, i quali hanno successivamente introdotto nel contesto italiano metodologie di lavoro, approcci innovativi e una forte attenzione alla concretezza nello sviluppo di soluzioni tecnologiche.

1.2 Il progetto

Il tirocinio si è svolto presso Eggon Srl ed è stato dedicato allo sviluppo di **EggLog Motion Core**, un ecosistema per la telemetria applicata al motorsport motociclistico. Il sistema si articola in un dispositivo *Embedded_G* per l'acquisizione dei dati a bordo del veicolo e in un'applicazione dedicata alla consultazione, alla visualizzazione e all'analisi delle informazioni raccolte durante le sessioni di guida stradale o in pista.

L'obiettivo del progetto è stato realizzare un *Firmware_G* affidabile per il dispositivo embedded, basato su microcontrollore ESP32 (vd. paragrafo ESP32), in grado di acquisire i dati provenienti dai diversi sensori installati a bordo, organizzarli in modo efficiente e memorizzarli localmente su scheda microSD, garantendo continuità di acquisizione, integrità dei dati e prestazioni adeguate ai vincoli tipici di un sistema real-time. Parallelamente, è stata sviluppata un'applicazione destinata all'interazione con il dispositivo e all'analisi

dei dati acquisiti.

Il lavoro ha previsto inoltre lo studio del prototipo realizzato dal precedente gruppo di tirocinanti, con l'obiettivo di valutare le soluzioni adottate e individuare possibili margini di miglioramento, sia a livello di architettura software sia in vista di una futura revisione hardware del dispositivo.

Il progetto ha permesso di affrontare problematiche caratteristiche dello sviluppo di sistemi embedded destinati ad applicazioni reali, approfondendo aspetti relativi all'acquisizione dei dati da sensori, alla gestione efficiente delle risorse hardware e alla progettazione di firmware affidabili.

1.3 Motivazioni della scelta del progetto

Le principali motivazioni che mi hanno portato a scegliere questo progetto di tirocinio sono:

- **Interesse per l'Internet of Things e i sistemi embedded**

Fin dagli anni della scuola secondaria di secondo grado ho sviluppato un forte interesse per il mondo dell'*Internet of Things_G*, grazie alla realizzazione di alcuni progetti che mi hanno permesso di avvicinarmi ai sistemi embedded. Questo interesse mi ha spinto a ricercare un'esperienza che mi consentisse di approfondire tali tematiche in un contesto professionale.

- **Interesse per l'automotive**

Nel corso degli anni ho sviluppato un particolare interesse per il settore *Automotive_G* e per le tecnologie che ne caratterizzano l'evoluzione, come i sistemi di telemetria, le reti di comunicazione veicolari e le applicazioni software dedicate all'acquisizione e all'analisi dei dati.

- **Ambiente di lavoro giovane**

Un ulteriore elemento che ha influenzato la mia scelta è stato il contesto aziendale. Essendo composta prevalentemente da un team giovane, l'azienda favorisce una comunicazione diretta, il confronto continuo e uno scambio di idee privo delle barriere.

- **Approfondimento dello sviluppo firmware**

Nel corso di precedenti esperienze avevo già utilizzato piattaforme come *Arduino_G*, ma esclusivamente per applicazioni relativamente semplici. Il tirocinio rappresentava quindi un'opportunità per affrontare uno sviluppo firmware più strutturato, lavorando direttamente su un prodotto reale e acquisendo competenze più avanzate nel campo dell'embedded.

- **Crescita personale**

Gli argomenti relativi alle telecomunicazioni sono sempre stati tra quelli che ho trovato più complessi durante il percorso scolastico. Per questo motivo ho visto il progetto come una sfida personale, con l'obiettivo di consolidare le mie conoscenze e migliorare le mie competenze.

1.4 Convenzioni tipografiche

Durante la redazione del presente documento sono state adottate alcune convenzioni tipografiche con l'obiettivo di rendere più chiara e uniforme la consultazione del testo. In particolare:

- Gli acronimi, le abbreviazioni e i termini tecnici o di uso non comune vengono descritti all'interno del *glossario*, riportato nella parte finale del documento;
- Alla prima occorrenza di un termine presente nel glossario viene utilizzata la seguente nomenclatura: *Arduino_G*;
- Nel caso degli acronimi alla prima ricorrenza viene utilizzato il riferimento per esteso (*Industrial Internet of Things_G*), mentre dalla seconda in poi viene semplicemente riportato come acronimo (IIoT);
- I termini in lingua straniera non entrati nell'uso comune della lingua italiana, così come quelli appartenenti al gergo tecnico, vengono riportati in carattere *corsivo*;
- I nomi di funzioni, variabili o altri elementi appartenenti al codice sorgente vengono rappresentati mediante carattere **monospaziato**;
- I riferimenti a libri, articoli o altre risorse presenti nella bibliografia (*bibliografia*) sono indicati tramite il relativo identificativo numerico, ad esempio *Manifesto Agile*. URL: <http://agilemanifesto.org/iso/it/>;
- Gli estratti di codice sorgente vengono riportati attraverso appositi blocchi formatati, come riportato di seguito:

```
def recur_fibo(n):  
    if n <= 1:  
        return n  
    else:  
        return(recur_fibo(n-1) + recur_fibo(n-2))  
nterms = 10  
if nterms <= 0:  
    print("Plese enter a positive integer")  
else:  
    print("Fibonacci sequence:")  
    for i in range(nterms):  
        print(recur_fibo(i))
```

Codice 1.1: Fibonacci recursive

2. Descrizione dello stage

Il presente capitolo descrive il percorso di stage svolto presso l'azienda, illustrando gli obiettivi formativi, le competenze richieste e le attività pianificate durante il periodo di tirocinio. Vengono inoltre presentate l'organizzazione del lavoro adottata, l'analisi preventiva dei principali rischi progettuali e il ruolo svolto dal tutor aziendale nel supporto alle attività di sviluppo.

2.1 Competenze e obiettivi

L'obiettivo principale dello stage è stato l'evoluzione e l'ottimizzazione del firmware embedded del sistema *EggLog Motion Core*, una piattaforma dedicata all'acquisizione e alla gestione di dati telemetrici in ambito motorsport, sviluppata su piattaforma ESP32.

Il lavoro riguarda principalmente l'analisi dell'architettura firmware esistente, il miglioramento dei componenti software già presenti e l'introduzione di nuove funzionalità volte a rendere il sistema più affidabile, efficiente e facilmente estendibile.

Le attività svolte hanno coinvolto diversi aspetti del sistema, tra cui la gestione dello *storage* su memoria SD, l'ottimizzazione dei servizi di comunicazione BLE (*Bluetooth Low Energy_G*) e Wi-Fi, la gestione del trasferimento dei dati acquisiti e l'analisi del traffico CAN proveniente dalla centralina elettronica (*Electronic Control Unit_G* detta ECU) della moto mediante tecniche di *CAN sniffing_G*.

2.1.1 Competenze da acquisire

Durante il periodo di tirocinio erano previste l'acquisizione e il consolidamento delle seguenti competenze:

- sviluppo e manutenzione di firmware embedded in linguaggio *C++_G* su piattaforma ESP32;
- analisi, *refactoring* e ottimizzazione di codice embedded esistente;
- gestione di sistemi di memorizzazione embedded e ottimizzazione delle operazioni di lettura e scrittura su memoria SD;
- sviluppo e ottimizzazione di servizi BLE e Wi-Fi;
- analisi dei protocolli di comunicazione utilizzati in ambito automotive, con particolare riferimento al *bus CAN_G*;
- acquisizione e analisi di messaggi dal bus CAN provenienti dalla ECU di un veicolo motorizzato;
- utilizzo di strumenti di *debugging* e *testing* per sistemi embedded;

- produzione di documentazione tecnica relativa al firmware e ai protocolli di comunicazione analizzati.

2.1.2 Obiettivi

Gli obiettivi definiti dall'azienda sono stati classificati secondo tre livelli di priorità:

- **O**: obbligatorio;
- **D**: desiderabile;
- **F**: facoltativo.

2.1.2.1 Obiettivi obbligatori

Gli obiettivi obbligatori comprendevano principalmente attività volte al miglioramento dell'affidabilità e delle prestazioni del firmware esistente.

- **OO-1**: *Refactoring* e ottimizzazione del sistema di lettura e scrittura su scheda SD;
- **OO-2**: Completamento del sistema di rotazione dei file con introduzione di meccanismi di recupero in caso di interruzioni impreviste;
- **OO-3**: Ottimizzazione delle procedure di download dei dati memorizzati sulla scheda SD;
- **OO-4**: Miglioramento dei servizi BLE e introduzione di nuovi comandi firmware;
- **OO-5**: Produzione della documentazione tecnica relativa al firmware sviluppato.

2.1.2.2 Obiettivi desiderabili

Gli obiettivi desiderabili erano principalmente orientati all'introduzione di funzionalità aggiuntive e al miglioramento delle modalità di trasferimento dei dati.

- **OD-1**: Implementazione del trasferimento dei dati tramite connessione Wi-Fi;
- **OD-2**: Introduzione di meccanismi di supporto alla ripresa dei trasferimenti interrotti;
- **OD-3**: Valutazione e introduzione di tecniche di compressione dei dati;
- **OD-4**: Integrazione dell'analisi e gestione dei dati provenienti dalla ECU;
- **OD-5**: Raccolta di metriche relative alle prestazioni dei trasferimenti dati con relativi *Benchmark_G* per il confronto.

2.1.2.3 Obiettivi facoltativi

- **OF-1**: Implementazione di un sistema avanzato di indicizzazione delle sessioni memorizzate sulla scheda SD.

2.2 Pianificazione

Le attività di stage sono state pianificate con il tutor aziendale considerando un monte complessivo di 320 ore, distribuite su otto settimane lavorative da quaranta ore ciascuna. Tali settimane sono state suddivise come segue:

1. Prima settimana – *Onboarding* e analisi del progetto

- Introduzione al sistema **EggLog Motion Core** e all'architettura generale della piattaforma;
- Configurazione dell'ambiente di sviluppo e degli strumenti di compilazione e debugging;
- Analisi della struttura del firmware esistente e delle principali componenti software;
- Studio delle modalità di acquisizione, memorizzazione e trasferimento dei dati.

2. Seconda settimana – Studio delle tecnologie e analisi del firmware

- Approfondimento della gestione della memoria microSD e del relativo *file system*;
- Analisi dello stack BLE e dei servizi implementati;
- Studio dei protocolli di comunicazione utilizzati dal sistema;
- Individuazione delle principali criticità presenti nel firmware.

3. Terza settimana – Analisi e progettazione delle modifiche

- Analisi dell'architettura software esistente e definizione degli interventi di *refactoring*;
- Progettazione delle modifiche relative alla gestione dello storage;
- Studio dei meccanismi di recupero e gestione dei dati in caso di errore;
- Analisi preliminare del bus CAN della moto e delle informazioni trasmesse dalla ECU.

4. Quarta settimana – Sviluppo e ottimizzazione dello storage

- *Refactoring* dei moduli responsabili della gestione della memoria SD;
- Ottimizzazione delle operazioni di lettura e scrittura dei dati;
- Miglioramento del sistema di rotazione dei file;
- Implementazione e verifica dei meccanismi di *recovery*.

5. Quinta settimana – Gestione dei dati e comunicazioni

- Implementazione delle funzioni di calibrazione dei sensori;
- Ottimizzazione delle procedure di download dei dati memorizzati;
- Studio delle possibilità di compressione dei dati;
- Miglioramento dei servizi BLE.

6. Sesta settimana – Analisi dei pacchetti CAN e integrazione firmware

- Acquisizione dei messaggi presenti sul bus CAN tramite attività di *sniffing*;
- Analisi degli identificativi e degli identificativi dei pacchetti CAN provenienti dalla ECU;
- Studio della possibile integrazione delle informazioni trasmesse nel bus CAN all'interno del sistema telemetrico;
- Evoluzione dei comandi e dei servizi firmware.

7. Settima settimana – Testing e validazione

- Esecuzione di test funzionali sul firmware modificato;
- Raccolta di metriche sulle prestazioni del sistema;
- Esecuzione di benchmark sulle operazioni di memorizzazione e trasferimento dati;
- Analisi e risoluzione delle problematiche emerse durante i test.

8. Ottava settimana – Documentazione finale

- Aggiornamento della documentazione tecnica del firmware;
- Descrizione delle modifiche architetturali introdotte;
- Documentazione relativa ai protocolli di comunicazione analizzati;
- Stesura della relazione finale di stage.

2.3 Organizzazione del lavoro

Le attività di sviluppo sono state organizzate seguendo una metodologia *Metodologia Agile*, ispirata al framework *Scrum*. Il lavoro è stato suddiviso in sprint della durata indicativa di due settimane, durante i quali venivano pianificate le attività da svolgere, definiti gli obiettivi da raggiungere e monitorato lo stato di avanzamento delle attività insieme al tutor aziendale.

Durante il periodo di stage sono stati inoltre svolti degli *Stand-up meeting* con cadenza giornaliera, finalizzati al mantenimento dell'allineamento tra i membri del team. Questi incontri hanno permesso di condividere i progressi effettuati, discutere eventuali problematiche emerse durante le fasi di sviluppo e progettazione e individuare possibili soluzioni attraverso il confronto tecnico.

Il gruppo di lavoro era composto da due tirocinanti, ciascuno responsabile dello sviluppo di una specifica componente del sistema. Il sottoscritto ha operato principalmente sulla componente firmware embedded, occupandosi dell'evoluzione del software a bordo del dispositivo, mentre la collega ha lavorato sulle componenti applicative del sistema.

Nonostante la suddivisione delle responsabilità, le attività sono state svolte in maniera collaborativa, favorendo il confronto continuo tra i membri del team e garantendo una corretta integrazione tra firmware, applicazione mobile e infrastruttura software.

2.4 Analisi preventiva dei rischi

Prima dell'avvio delle attività è stata effettuata un'analisi preventiva dei principali rischi che avrebbero potuto influenzare il corretto svolgimento del progetto.

- **Comprensione del firmware esistente**

la necessità di intervenire su un sistema software già sviluppato ha richiesto una fase iniziale di analisi dell'architettura firmware e dei meccanismi implementati, al fine di ridurre il rischio di introdurre regressioni durante le modifiche.

- **Gestione dello storage e integrità dei dati**

le operazioni di lettura, scrittura e trasferimento dei dati dalla memoria micro-SD rappresentavano una possibile criticità in termini di affidabilità. Il rischio è stato mitigato attraverso attività di *testing*, analisi delle prestazioni e utilizzo di *benchmark*.

- **Analisi del traffico CAN**

l'acquisizione e l'interpretazione dei dati provenienti dalla ECU della moto rappresentavano una possibile criticità, in quanto i messaggi presenti sul bus CAN non seguono necessariamente una codifica standardizzata.

Inoltre, prima dell'attività di *sniffing*, è stata quindi svolta un'analisi di fattibilità sia dal punto di vista hardware, valutando l'interfacciamento fisico con il veicolo, sia dal punto di vista software, considerando l'acquisizione, l'elaborazione e l'interpretazione dei dati raccolti.

- **Coordinamento tra le componenti del sistema**

la presenza di più componenti sviluppate parallelamente poteva introdurre problematiche di integrazione tra firmware, applicazione mobile e infrastruttura software. Tale rischio è stato mitigato attraverso confronti periodici con il tutor aziendale, revisioni del codice e attività di integrazione condivise.

3. Analisi dei requisiti

Questo capitolo presenta gli use case rilevanti per le attività di ottimizzazione ed evoluzione svolte sul firmware embedded del dispositivo EggLog Motion Core (d'ora in poi solo EggLog) durante il periodo di tirocinio, insieme alla relativa analisi dei requisiti.

3.1 Stato attuale del firmware

Allo stato attuale, il firmware di EggLog è in grado di acquisire i dati provenienti dai sensori di bordo (*IMU_G*, magnetometro, barometro e modulo GPS). I dati acquisiti vengono temporaneamente memorizzati in buffer in *RAM_G* secondo uno schema di *Ping-pong buffering_G* e successivamente scritti in formato testuale (*CSV_G*) su scheda microSD, organizzati in file distinti all'interno di cartelle dedicate a ciascun sensore.

Il dispositivo espone inoltre un web server locale, raggiungibile via WiFi, che consente all'utente di configurare i parametri di funzionamento, scaricare i dati archiviati in un intervallo temporale specifico e rimuoverli dalla scheda microSD.

3.2 Analisi degli utenti

Prima di affrontare le fasi di progettazione e sviluppo, è opportuno individuare a chi si rivolge il prodotto finale, soprattutto quando questo non è ancora disponibile sul mercato. L'azienda ha già condotto uno studio preliminare del pubblico di riferimento, individuando tre principali categorie di utenti:

- i **centri di *coaching***, che possono impiegare i dati sensoriali raccolti come supporto didattico per il miglioramento tecnico dei piloti.
- i **team organizzatori di eventi motociclistici**, interessati a raccogliere dati telemetrici dettagliati sui partecipanti alle gare;
- i **piloti semi-professionisti**, che possono sfruttare EggLog per valutare le proprie prestazioni durante sessioni di allenamento o competizioni ufficiali;

3.3 Lista degli *use case*

Per facilitare la comprensione dei casi d'uso, questi saranno descritti da un diagramma ciascuno, che rispetta la sintassi *UML_G*. La descrizione testuale deve contenere le seguenti informazioni:

- **Attore**: rappresenta il ruolo che un'entità esterna al sistema assume quando interagisce con esso per raggiungere un obiettivo.

- **Precondizioni:** condizioni che devono essere vere nello stato del sistema prima che il caso d'uso inizi la sua esecuzione.
- **Postcondizioni:** condizioni che devono essere vere nello stato del sistema dopo che il caso d'uso è terminato.
- **Scenario principale:** interazioni tra attore e sistema che portano al raggiungimento dell'obiettivo del caso d'uso con successo.
- **Scenari alternativi:** variazioni rispetto al flusso principale.
- **Inclusioni:** riferimenti ai casi d'uso inclusi.
- **Estensioni:** riferimenti ai casi d'uso di estensione.

Di seguito sono riportati esclusivamente gli use case relativi alle attività di ottimizzazione ed evoluzione effettivamente svolte durante il tirocinio.

3.3.1 UC1 – Acquisizione binaria dei dati sensore

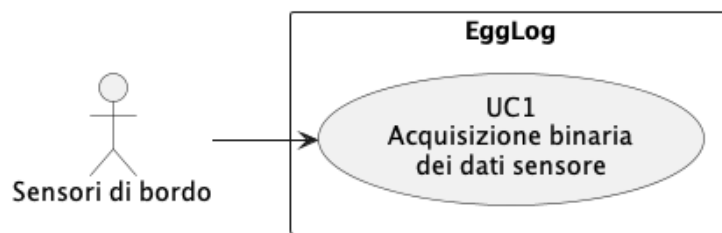


Figura 3.1: Diagramma UML – UC1: Acquisizione binaria dei dati sensore

Attore: sensori di bordo.

Precondizioni:

- I sensori sono stati inizializzati e calibrati (cfr. [UC7](#));
- È in corso una sessione di registrazione attiva.

Postcondizioni:

- Il buffer in RAM contiene i campioni acquisiti, serializzati in formato binario, pronti per essere scritti su SD.

Scenario principale:

1. Il firmware richiede un nuovo campione a ciascun sensore attivo, secondo la frequenza di campionamento configurata (cfr. [UC5](#));
2. ciascun sensore restituisce il valore misurato;
3. il firmware associa a ogni campione il timestamp corrente;
4. il firmware serializza il campione in formato binario nativo;
5. il firmware inserisce il campione serializzato nel buffer in RAM;

6. se il buffer ha raggiunto la soglia di scrittura, il firmware avvia UC2.

Scenari alternativi:

- 2a. un sensore non risponde entro il timeout previsto: il campione viene marcato come non valido e l'acquisizione prosegue con i restanti sensori.

Inclusioni: nessuna.

Estensioni: nessuna.

3.3.2 UC2 – Registrazione binaria dei dati sensore su SD

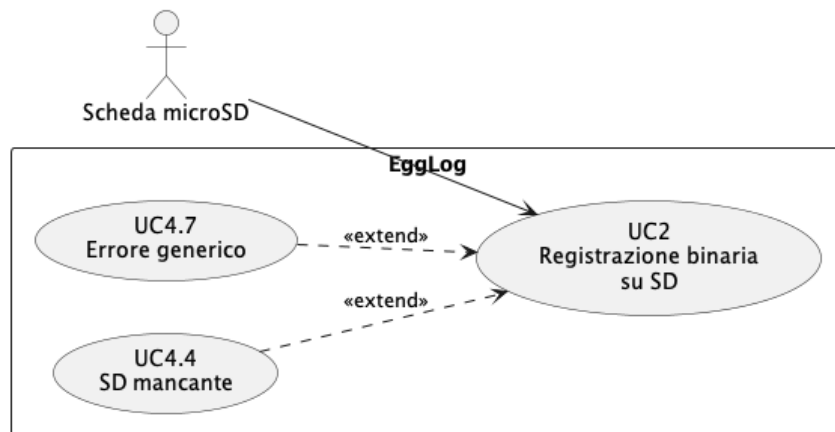


Figura 3.2: Diagramma UML – UC2: Registrazione binaria dei dati sensore su SD

Attore: scheda microSD.

Precondizioni:

- Il buffer di acquisizione (cfr. UC1) contiene almeno un pacchetto di dati binari pronto per la scrittura;
- la scheda microSD è presente e correttamente montata.

Postcondizioni:

- I dati sono persistiti sulla scheda microSD in formato binario e il buffer in RAM viene svuotato.

Scenario principale:

1. Il firmware verifica che il buffer abbia raggiunto la soglia di scrittura;
2. il firmware accede al file su scheda microSD associato al sensore a cui appartiene il buffer;
3. il firmware scrive il contenuto del buffer sul file in formato binario;
4. il firmware svuota il buffer in RAM;
5. il firmware conferma l'avvenuta scrittura.

Scenari alternativi:

- 1a. la scheda microSD non è presente: la scrittura viene rimandata e i dati restano nel buffer fino a un numero massimo di tentativi;
- 3a. la scrittura fallisce (SD piena o guasta): il firmware segnala l'errore e applica una politica di retry.

Inclusioni: nessuna.

Estensioni: [UC4.4](#), [UC4.7](#).

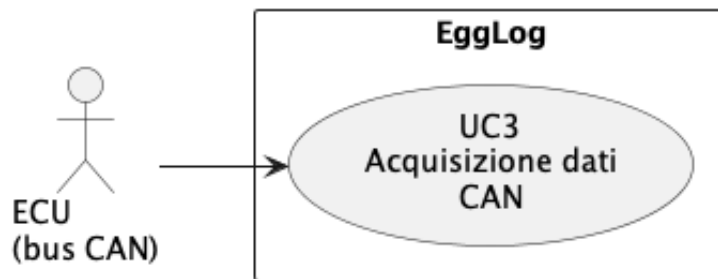
3.3.3 UC3 – Acquisizione dati dal bus CAN

Figura 3.3: Diagramma UML – UC3: Acquisizione dati dal CAN

Attore: ECU del veicolo mediante bus CAN.

Precondizioni:

- Il modulo di interfaccia CAN è collegato al bus dati del veicolo;
- è in corso una sessione di registrazione attiva.

Postcondizioni:

- I dati ECU acquisiti sono timestampati, sincronizzati con gli altri sensori e resi disponibili per la registrazione (cfr. [UC2](#)).

Scenario principale:

1. Il firmware si mette in ascolto sul bus CAN;
2. la ECU trasmette periodicamente i messaggi contenenti i parametri motore;
3. il firmware riceve e decodifica ciascun messaggio CAN;
4. il firmware associa a ciascun dato il timestamp corrente;
5. il firmware serializza i dati in formato binario, coerentemente con [UC1](#);
6. il firmware inserisce i dati nel buffer di acquisizione.

Scenari alternativi:

- 3a. il messaggio CAN ricevuto non è riconosciuto o risulta malformato: il messaggio viene scartato.

Inclusioni: nessuna.

Estensioni: nessuna.

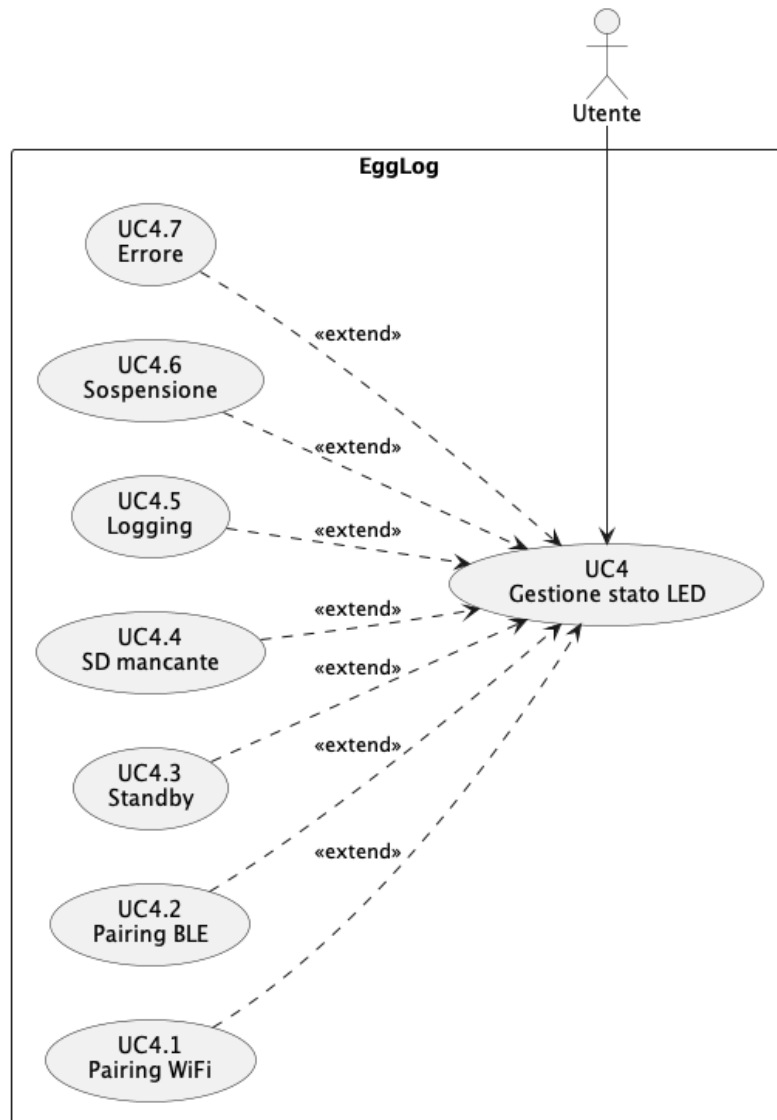
3.3.4 UC4 – Gestione dello stato del firmware tramite *LED_G*

Figura 3.4: Diagramma UML – UC4: Gestione dello stato del firmware tramite LED

Attore: utente.**Precondizioni:**

- Il dispositivo è alimentato e il firmware è in esecuzione.

Postcondizioni:

- Il LED riflette lo stato operativo corrente del dispositivo.

Scenario principale:

1. Il firmware rileva un cambiamento nello stato operativo del dispositivo;
2. il firmware seleziona il pattern di colore/lampeggio associato al nuovo stato;
3. il firmware applica il pattern al LED;

4. l'utente osserva il LED e ne interpreta lo stato del dispositivo.

Scenari alternativi:

- 1a. si verificano più condizioni contemporaneamente: il firmware seleziona il pattern relativo alla condizione a priorità più alta.

Inclusioni: nessuna.

Estensioni: UC4.1, UC4.2, UC4.3, UC4.4, UC4.5, UC4.6, UC4.7.

3.3.4.1 UC4.1 – Segnalazione di pairing WiFi

Attore: utente.

Precondizioni:

- È in corso una procedura di *pairing* con una rete WiFi.

Postcondizioni:

- Il LED mostra il pattern blu con lampeggio veloce (circa 4 Hz).

Scenario principale:

1. Il firmware avvia la procedura di pairing WiFi;
2. il modulo di gestione del LED applica il pattern blu con lampeggio veloce;
3. il pattern rimane attivo per l'intera durata del pairing;
4. al termine della procedura, con esito positivo o negativo, il firmware richiama UC4 per applicare il pattern corrispondente al nuovo stato.

Scenari alternativi: nessuno.

Inclusioni: nessuna.

Estensioni: nessuna.

3.3.4.2 UC4.2 – Segnalazione di pairing BLE

Attore: utente.

Precondizioni:

- È in corso una procedura di associazione (*pairing*) con un dispositivo tramite BLE.

Postcondizioni:

- Il LED mostra il pattern blu con lampeggio lento (circa 1 Hz).

Scenario principale:

1. Il firmware avvia la procedura di pairing BLE;
2. il modulo di gestione del LED applica il pattern blu con lampeggio lento;
3. il pattern rimane attivo per l'intera durata del pairing;

4. al termine della procedura, con esito positivo o negativo, il firmware richiama UC4 per applicare il pattern corrispondente al nuovo stato.

Scenari alternativi: nessuno.

Inclusioni: nessuna.

Estensioni: nessuna.

3.3.4.3 UC4.3 – Segnalazione di dispositivo avviato in attesa di sessione

Attore: utente.

Precondizioni:

- Il dispositivo è stato avviato correttamente ma non ha ancora avviato alcuna sessione di registrazione.

Postcondizioni:

- Il LED mostra il pattern bianco fisso.

Scenario principale:

1. Il firmware completa la fase di avvio senza avviare alcuna sessione;
2. il modulo di gestione del LED applica il pattern bianco fisso;
3. il pattern rimane attivo fino all'avvio di una sessione di registrazione;
4. all'avvio della sessione, il firmware richiama UC4 per applicare il pattern di logging normale.

Scenari alternativi: nessuno.

Inclusioni: nessuna.

Estensioni: nessuna.

3.3.4.4 UC4.4 – Segnalazione di scheda microSD mancante

Attore: utente.

Precondizioni:

- La scheda microSD risulta assente.

Postcondizioni:

- Il LED mostra il pattern arancione con lampeggio regolare (circa 1 Hz)

Scenario principale:

1. Il firmware rileva l'assenza della scheda microSD;
2. il modulo di gestione del LED applica il pattern arancione;
3. il pattern rimane attivo finché la scheda microSD non viene rilevata correttamente;
4. alla rilevazione della scheda microSD, il firmware richiama UC4 per applicare il pattern corrispondente allo stato operativo corrente.

Scenari alternativi: nessuno.

Inclusioni: nessuna.

Estensioni: nessuna.

3.3.4.5 UC4.5 – Segnalazione di logging normale

Attore: utente.

Precondizioni:

- È in corso una regolare sessione di registrazione dati.

Postcondizioni:

- Il LED mostra il pattern verde fisso.

Scenario principale:

1. Il firmware avvia o riprende la sessione di registrazione;
2. il modulo di gestione del LED applica il pattern verde fisso;
3. il pattern rimane attivo per l'intera durata della sessione;
4. non appena la sessione termina o viene sospesa, il firmware richiama UC4 per applicare il pattern corrispondente al nuovo stato.

Scenari alternativi: nessuno.

Inclusioni: nessuna.

Estensioni: nessuna.

3.3.4.6 UC4.6 – Segnalazione di sospensione per inattività

Attore: utente.

Precondizioni:

- Si verifica la sospensione automatica dell'acquisizione sensori descritta in UC6.

Postcondizioni:

- Il LED mostra il pattern verde pulsante lento (effetto *breathing*, circa 0.5 Hz).

Scenario principale:

1. UC6 sospende l'acquisizione dei sensori non essenziali;
2. il modulo di gestione del LED applica il pattern verde pulsante lento;
3. il pattern rimane attivo finché l'acquisizione sensori resta sospesa;
4. al rilevamento di un nuovo movimento, il firmware richiama UC4 per ripristinare il pattern di logging normale.

Scenari alternativi: nessuno.

Inclusioni: nessuna.

Estensioni: nessuna.

3.3.4.7 UC4.7 – Segnalazione di errore generico

Attore: utente.

Precondizioni:

- Si verifica una condizione di errore o malfunzionamento generica.

Postcondizioni:

- Il LED mostra il pattern rosso con lampeggio veloce (circa 4 Hz).

Scenario principale:

1. Il firmware rileva una condizione di errore;
2. il modulo di gestione del LED applica il pattern rosso con lampeggio veloce, con priorità su qualunque altro stato, incluso quello di microSD mancante;
3. il pattern rimane attivo finché la condizione di errore non viene risolta o il dispositivo riavviato;
4. non appena la condizione di errore viene risolta, il firmware richiama [UC4](#) per applicare il pattern corrispondente allo stato operativo corrente.

Scenari alternativi: nessuno.

Inclusioni: nessuna.

Estensioni: nessuna.

3.3.5 UC5 – Configurazione dell'*Output Data Rate_G* (ODR) dei sensori

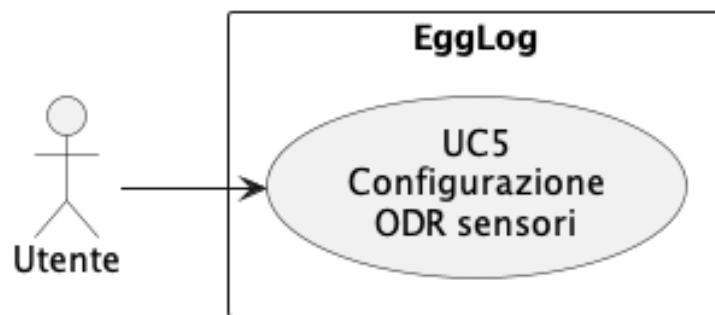


Figura 3.5: Diagramma UML – UC5: Configurazione dell'ODR dei sensori

Attore: utente.

Precondizioni:

- Il dispositivo è raggiungibile per la configurazione (es. tramite pairing BLE già stabilito cfr. [UC4.2](#)).

Postcondizioni:

- La nuova frequenza di campionamento è salvata in memoria persistente e applicata alle acquisizioni successive.

Scenario principale:

1. L'utente richiede la modifica della frequenza di campionamento;
2. il firmware riceve il nuovo valore richiesto;
3. il firmware salva la nuova configurazione in memoria persistente;
4. il firmware applica la nuova frequenza alle acquisizioni successive.

Scenari alternativi: nessuno.

Inclusioni: nessuna.

Estensioni: nessuna.

3.3.6 UC6 – Sospensione dell'acquisizione sensori in stato di inattività

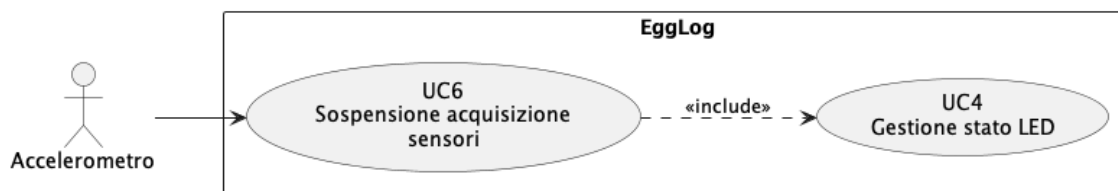


Figura 3.6: Diagramma UML – UC6: Sospensione dell'acquisizione sensori in stato di inattività

Attore: accelerometro.

Precondizioni:

- È in corso una sessione di registrazione attiva.

Postcondizioni:

- L'acquisizione dei sensori non essenziali risulta sospesa o riattivata in base allo stato di movimento rilevato.

Scenario principale:

1. Il firmware monitora costantemente i dati dell'accelerometro;
2. il firmware verifica se non si sono registrate variazioni significative per l'intervallo di tempo configurato;
3. al superamento dell'intervallo, il firmware sospende l'acquisizione dei sensori non essenziali;
4. il firmware richiama [UC4](#) per applicare il pattern LED di sospensione (cfr. [UC4.6](#));
5. il firmware continua a monitorare l'accelerometro per rilevare un nuovo movimento;
6. al rilevamento di movimento, il firmware riattiva l'acquisizione e applica [UC4](#) per ripristinare il pattern di logging normale.

Scenari alternativi:

- 1a. la sospensione automatica è stata disabilitata dall'utente: il firmware non sospende mai l'acquisizione, indipendentemente dallo stato di movimento rilevato.

Inclusioni: UC4.

Estensioni: nessuna.

3.3.7 UC7 – Calibrazione dei sensori

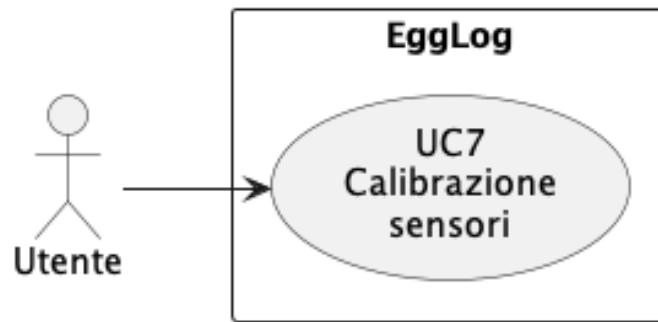


Figura 3.7: Diagramma UML – UC7: Calibrazione dei sensori

Attore: utente.

Precondizioni:

- Il dispositivo è alla prima configurazione oppure non esiste una calibrazione valida in memoria per la sessione corrente.

Postcondizioni:

- I parametri di calibrazione sono salvati in memoria persistente e applicati alle letture sensore successive.

Scenario principale:

1. Il firmware rileva l'assenza di una calibrazione valida;
2. il firmware avvia la procedura di calibrazione, richiedendo che il dispositivo esegua determinati movimenti;
3. il firmware calcola gli offset di calibrazione;
4. il firmware salva i parametri di calibrazione in memoria persistente;
5. il firmware applica i parametri di calibrazione alle acquisizioni successive (cfr. UC1).

Scenari alternativi: nessuno.

Inclusioni: nessuna.

Estensioni: nessuna.

3.4 Tracciamento dei requisiti

In questa sezione si riporta la lista di requisiti rilevata in seguito all'analisi dell'utenza e degli use case; i requisiti sono identificati da un codice definito con la seguente nomenclatura:

R-(Tipo)-(Urgenza)-(Codice)

Dove:

- **Tipo:** F (Funzionale), N (Non funzionale) e V (Vincolo);
- **Urgenza:** Ob (Obbligatoria), De (Desiderabile) e Op (Opzionale);
- **Codice:** numero progressivo del requisito.

3.4.1 Requisiti funzionali

Nella Tabella 3.1 si riporta il tracciamento dei requisiti funzionali, ovvero le funzionalità e i servizi che il sistema deve offrire per essere conforme alle aspettative.

Codice	Descrizione	Fonti
R-F-Ob-1	EggLog deve acquisire e serializzare in formato binario i dati provenienti dai sensori, riducendo la dimensione del buffer in RAM	UC1
R-F-Ob-2	EggLog deve scrivere su scheda microSD i dati acquisiti in formato binario	UC2
R-F-Ob-3	EggLog deve leggere i dati esposti dalla centralina motore (ECU) tramite bus CAN	UC3
R-F-Ob-4	EggLog deve eseguire una procedura di calibrazione dei sensori a inizio sessione o alla prima configurazione del dispositivo	UC7
R-F-De-5	EggLog deve pilotare il LED integrato sulla scheda ESP32-S3 per segnalare, tramite pattern di colore e lampeggio distinti, lo stato corrente del dispositivo	UC4, UC4.1, UC4.2, UC4.3, UC4.4, UC4.5, UC4.6, UC4.7
R-F-De-6	L'utente deve poter configurare l'output rate (ODR) dei sensori nell'intervallo 10-100 Hz	UC5
R-F-De-7	EggLog deve sospendere automaticamente l'acquisizione dei sensori non essenziali in stato di inattività prolungata e riattivarla al rilevamento di movimento	UC6
Continua nella prossima pagina...		

Tabella 3.1 – Continuo della tabella

Codice	Descrizione	Fonti
--------	-------------	-------

Tabella 3.1: Tracciamento dei requisiti funzionali.

3.4.2 Requisiti non funzionali

Nella Tabella 3.2 si riporta il tracciamento dei requisiti non funzionali, ovvero l'insieme di criteri quantificabili di qualità che il sistema sviluppato deve rispettare per essere conforme alle aspettative e agli obiettivi preposti.

Codice	Descrizione	Fonti
R-N-Ob-1	Il consumo di RAM per il buffering dei dati sensore deve essere ridotto rispetto al formato testuale grazie alla serializzazione binaria	UC1
R-N-Ob-2	Il consumo energetico del dispositivo deve essere ridotto rispetto alla configurazione precedente grazie all'acquisizione/scrittura binaria e alla sospensione automatica in stato di inattività	UC1, UC6
R-N-Db-3	Il cambio di stato segnalato dal LED deve essere percepito come pressoché istantaneo dall'utente, e ogni pattern di colore/lampeggio deve risultare univocamente distinguibile dagli altri	UC4, UC4.1, UC4.2, UC4.3, UC4.4, UC4.5, UC4.6, UC4.7
R-N-Db-4	L'accuratezza delle misurazioni successive alla calibrazione deve rientrare entro le tolleranze specificate dal produttore dei sensori	UC7

Tabella 3.2: Tracciamento dei requisiti non funzionali.

3.4.3 Requisiti di vincolo

Nella Tabella 3.3 si riporta il tracciamento dei requisiti di vincolo, ovvero l'insieme delle limitazioni tecnologico-organizzative a cui il dispositivo deve sottostare per essere conforme alle aspettative e alle esigenze degli stakeholder.

Codice	Descrizione	Fonti
R-V-Ob-1	EggLog del dispositivo deve appartenere alla famiglia ESP32, nello specifico ESP32-S3	Tutti
R-V-Ob-2	Il firmware del dispositivo dev'essere scritto in linguaggio C/C++	Tutti
R-V-Ob-3	L'interfaccia con il bus CAN del veicolo deve essere compatibile con il protocollo CAN utilizzato dalla centralina (ECU) della moto	UC3
R-V-Ob-4	Il LED utilizzato per la segnalazione di stato deve essere quello integrato sulla scheda ESP32-S3	UC4

Tabella 3.3: Tracciamento dei requisiti di vincolo.

3.4.4 Riepilogo dei requisiti

Nella Tabella 3.4 si riporta il numero di requisiti per categoria e per urgenza; in totale sono stati rilevati 15 requisiti.

	Obbligatoria	Desiderabili	Facoltativi	TOTALE
RF	4	3	0	7
RN	2	2	0	4
RV	4	0	0	4
TOTALE	10	5	0	15

Tabella 3.4: Conteggio dei requisiti tracciati.

4. Tecnologie utilizzate

5. Progettazione e codifica

Breve introduzione al capitolo

5.1 Tecnologie e strumenti

Di seguito viene data una panoramica delle tecnologie e strumenti utilizzati.

Tecnologia 1

Descrizione Tecnologia 1.

Tecnologia 2

Descrizione Tecnologia 2

5.2 Ciclo di vita del software

5.3 Progettazione

5.3.1 Namespace 1

Descrizione namespace 1.

5.4 Design Pattern utilizzati

5.5 Codifica

6. Verifica e validazione

7. Conclusioni

7.1 Consuntivo finale

7.2 Raggiungimento degli obiettivi

7.3 Conoscenze acquisite

7.4 Valutazione personale

Glossario

Metodologia Agile La metodologia Agile è un approccio alla gestione e allo sviluppo di progetti software basato su iterazioni brevi, collaborazione continua e capacità di adattarsi rapidamente ai cambiamenti dei requisiti. Le attività vengono organizzate in cicli di sviluppo incrementali, durante i quali il team pianifica, realizza e verifica progressivamente le funzionalità del sistema, favorendo il confronto tra i membri del gruppo e il miglioramento continuo del prodotto.. [7](#)

Arduino Arduino è una piattaforma hardware e software open-source basata su microcontrollori, progettata per facilitare lo sviluppo e la prototipazione di sistemi elettronici e applicazioni embedded.. [2](#), [3](#)

Automotive Il termine automotive si riferisce al settore industriale e tecnologico dedicato alla progettazione, allo sviluppo, alla produzione e alla gestione dei veicoli a motore e delle tecnologie ad essi associate.. [2](#)

Benchmark Un benchmark è una procedura di valutazione delle prestazioni di un sistema, un'applicazione o un componente software attraverso l'esecuzione di specifici test e la raccolta di metriche quantitative. In ambito informatico viene utilizzato per confrontare diverse soluzioni, individuare eventuali colli di bottiglia e verificare l'efficacia delle ottimizzazioni apportate.. [5](#)

Bluetooth Low Energy Bluetooth Low Energy è una tecnologia di comunicazione wireless a basso consumo energetico basata sullo standard Bluetooth. È progettata per applicazioni che richiedono lo scambio periodico di piccole quantità di dati mantenendo un ridotto consumo energetico, risultando particolarmente adatta per dispositivi embedded, sensori e sistemi IoT.. [4](#)

bus CAN Il bus Controller Area Network (CAN) è una rete di comunicazione seriale progettata per consentire lo scambio affidabile di dati tra dispositivi elettronici all'interno di un sistema distribuito. È ampiamente utilizzato in ambito automotive per la comunicazione tra le centraline elettroniche del veicolo, grazie alla sua robustezza, alla gestione degli errori integrata e alla capacità di operare in ambienti caratterizzati da elevati livelli di disturbo elettromagnetico.. [4](#)

CAN sniffing Il CAN sniffing è una tecnica di analisi del traffico presente su una rete Controller Area Network (CAN) che consiste nell'acquisizione e registrazione dei messaggi scambiati tra i dispositivi collegati al bus. Questa attività permette di analizzare gli identificativi dei messaggi (*CAN ID*), i dati trasmessi e la frequenza di comunicazione, al fine di individuare la struttura e il significato delle informazioni scambiate tra le centraline elettroniche. In ambito automotive viene utilizzata, ad

esempio, per attività di diagnostica, sviluppo di sistemi telemetrici e analisi del comportamento delle ECU.. [4](#)

C++ C++ è un linguaggio di programmazione general-purpose derivato dal linguaggio C, che integra il supporto alla programmazione orientata agli oggetti e a diversi paradigmi di sviluppo. Grazie alle sue elevate prestazioni e alla possibilità di interagire direttamente con le risorse hardware, viene frequentemente utilizzato nello sviluppo di sistemi embedded e applicazioni dove sono richiesti efficienza e controllo delle risorse.. [4](#)

CSV CSV (*Comma-Separated Values*) è un formato di file testuale utilizzato per rappresentare dati tabellari attraverso una sequenza di valori separati da un delimitatore, generalmente una virgola. Questo formato è ampiamente utilizzato per lo scambio e l'archiviazione di dati strutturati grazie alla sua semplicità e alla compatibilità con numerosi strumenti di elaborazione e analisi dei dati.. [9](#)

Digital Health Con Digital Health si intende l'insieme di tecnologie e soluzioni digitali applicate al settore sanitario, che mirano a migliorare la gestione della salute, la prevenzione delle malattie e l'efficienza dei servizi sanitari. Include strumenti come dispositivi indossabili, applicazioni mobili, telemedicina, intelligenza artificiale e analisi dei dati per supportare la diagnosi, il monitoraggio e il trattamento dei pazienti.. [1](#)

Electronic Control Unit Una Electronic Control Unit (ECU) è una centralina elettronica presente nei veicoli moderni incaricata di controllare e gestire specifiche funzionalità del mezzo, come il controllo del motore, la gestione dell'iniezione o l'acquisizione di dati provenienti dai sensori. Le ECU comunicano frequentemente tra loro attraverso reti interne del veicolo, come il bus CAN, per lo scambio delle informazioni necessarie al funzionamento dei sistemi elettronici.. [4](#)

Embedded Un sistema embedded è un sistema hardware-software dedicato all'esecuzione di funzioni specifiche all'interno di un dispositivo elettronico, spesso caratterizzato da vincoli di prestazioni, memoria e consumo energetico.. [1](#)

Fintech Con Fintech si intende l'insieme di tecnologie e soluzioni digitali applicate al settore finanziario, che mirano a migliorare l'efficienza dei processi, a ridurre i costi e a offrire servizi più personalizzati ai clienti.. [1](#)

Firmware Software a basso livello che viene eseguito direttamente su un dispositivo hardware, permettendone il controllo e la gestione delle sue componenti elettroniche.. [1](#)

Industrial Internet of Things Con Industrial Internet of Things (IIoT) si intende l'insieme di tecnologie e dispositivi connessi a Internet che vengono utilizzati in ambito industriale per migliorare l'efficienza, la produttività e la sicurezza dei processi produttivi. L'IIoT consente la raccolta e l'analisi dei dati in tempo reale, permettendo alle aziende di prendere decisioni più informate e ottimizzare le operazioni.. [vii](#), [1](#), [3](#)

Internet of Things Insieme di dispositivi fisici connessi alla rete in grado di raccogliere, elaborare e scambiare dati, permettendo il monitoraggio e il controllo remoto delle informazioni.. [2](#)

IMU IMU (*Inertial Measurement Unit*) è un dispositivo elettronico composto generalmente da accelerometri e giroscopi, in grado di misurare rispettivamente l'accelerazione lineare e la velocità angolare di un oggetto. Le IMU vengono utilizzate per rilevare il movimento, l'orientamento e le variazioni di posizione di un dispositivo, risultando fondamentali in numerose applicazioni come sistemi embedded, robotica, veicoli e sistemi di telemetria.. [9](#)

Industria 5.0 Con Industria 5.0 si intende la quinta generazione di industrializzazione, caratterizzata dall'integrazione di tecnologie digitali, l'Intelligenza Artificiale e l'Automazione, per migliorare la produttività e l'efficienza dei processi produttivi.. [1](#)

Insurtech Con Insurtech si intende l'insieme di tecnologie e soluzioni digitali applicate al settore assicurativo, che mirano a migliorare l'efficienza dei processi, a ridurre i costi e a offrire servizi più personalizzati ai clienti.. [1](#)

LED Il LED (*Light Emitting Diode*) è un componente elettronico a semiconduttore in grado di emettere luce quando è attraversato da una corrente elettrica. Grazie al basso consumo energetico, alla lunga durata e all'elevata efficienza luminosa, è ampiamente utilizzato come indicatore di stato e per applicazioni di illuminazione in dispositivi elettronici ed embedded.. [v](#), [13](#)

Output Data Rate Con Output Data Rate (ODR) si indica la frequenza con cui un sensore acquisisce e rende disponibili nuovi dati in uscita. Generalmente è espressa in hertz (Hz) e determina il numero di campioni prodotti nell'unità di tempo. La scelta dell'ODR influisce sulla quantità di dati raccolti, sulla precisione temporale delle misurazioni e sul consumo energetico del dispositivo.. [v](#), [17](#)

Ping-pong buffering Il ping-pong buffering è una tecnica di gestione della memoria utilizzata nei sistemi embedded e nelle applicazioni che richiedono un'acquisizione continua di dati. Consiste nell'utilizzo di due buffer distinti: mentre un buffer viene utilizzato per la scrittura dei nuovi dati acquisiti, l'altro viene elaborato o trasferito dal sistema. Al termine di un ciclo di acquisizione, i due buffer vengono scambiati, permettendo di ridurre i tempi di attesa e garantire un flusso continuo di dati senza interrompere le operazioni di lettura o elaborazione.. [9](#)

RAM La RAM (*Random Access Memory*) è una memoria volatile utilizzata dal sistema per memorizzare temporaneamente dati e istruzioni necessari all'esecuzione dei programmi. Il suo contenuto viene perso allo spegnimento del dispositivo, ma consente un accesso rapido alle informazioni durante il funzionamento del sistema.. [9](#)

Scrum Scrum è un framework appartenente alla metodologia Agile utilizzato per organizzare il lavoro attraverso iterazioni chiamate sprint. Prevede ruoli, eventi e strumenti

specifici per pianificare le attività, monitorare l'avanzamento del progetto e favorire la collaborazione tra i membri del team.. [7](#)

Stand-up meeting Lo stand-up meeting è una breve riunione periodica utilizzata nelle metodologie Agile per favorire l'allineamento tra i membri di un team di sviluppo. Durante l'incontro ogni partecipante condivide lo stato delle attività svolte, gli obiettivi da completare e gli eventuali problemi o impedimenti riscontrati. L'obiettivo principale è garantire una comunicazione efficace, individuare tempestivamente eventuali criticità e coordinare il lavoro del gruppo.. [7](#)

UML UML (*Unified Modeling Language*) è un linguaggio di modellazione standard utilizzato nell'ingegneria del software per rappresentare graficamente la struttura, il comportamento e le interazioni di un sistema. Attraverso diversi tipi di diagrammi, UML supporta le attività di analisi, progettazione e documentazione del software, facilitando la comunicazione tra gli sviluppatori e gli stakeholder coinvolti.. [9](#)

Bibliografia

Testi

James P. Womack, Daniel T. Jones. *Lean Thinking, Second Editon*. Simon & Schuster, Inc., 2010 (cit. a p. [vii](#)).

Articoli

Einstein, Albert, Boris Podolsky e Nathan Rosen. «Can Quantum-Mechanical Description of Physical Reality be Considered Complete?» In: *Physical Review* 47.10 (1935), pp. 777–780. DOI: [10.1103/PhysRev.47.777](#) (cit. a p. [vii](#)).

Sitografia

Manifesto Agile. URL: <http://agilemanifesto.org/iso/it/> (cit. alle pp. 3, vii).

Questa pagina verrà momentaneamente utilizzata per vedere gli esempi di utilizzo di figure, tabelle, glossario e citazioni ecc...

Glossario

Esempio di utilizzo di un termine nel glossario *Industrial Internet of Things_G*. La prima volta che usi un acronimo usa "acronimo" per vedere l'acronimo esteso, da lì in poi usa la versione "acronimo".

Citazioni

Esempio di citazione direttamente nel testo *Manifesto Agile*.

Citazioni a piè pagina

Esempio di citazione nel piè di pagina¹.

Introduzione all'idea dello stage².

¹Daniel T. Jones James P. Womack. *Lean Thinking, Second Edition*. Simon & Schuster, Inc., 2010.

²Albert Einstein, Boris Podolsky e Nathan Rosen. «Can Quantum-Mechanical Description of Physical Reality be Considered Complete?» In: *Physical Review* 47.10 (1935), pp. 777–780. DOI: [10.1103/PhysRev.47.777](https://doi.org/10.1103/PhysRev.47.777).